



rich_

CSDN @凌小添

在编写代码的过程中，我们经常需要在终端中输出各种信息。传统的`print()`函数虽然简单易用，但在呈现复杂数据、美化输出格式方面却捉襟见肘。当你尝试打印一个嵌套层级很深的复杂字典、需要以表格形式展示数据，或者想要在终端中使用颜色和样式来突出重要信息时，`print()`函数的局限性就会变得非常明显。

这时，你可能会这样思考：

- "如何在终端中展示更美观的输出？"
- "有没有办法让我的调试信息更加清晰？"
- "如何在不使用GUI的情况下，让终端输出变得丰富多彩？"

这正是Rich库诞生的背景，今天我们深入了解这个能让输出焕然一新的工具。

1. Rich库

Rich是一个Python 库，专为美化终端输出而设计，它让你能够在终端中轻松添加颜色、样式、表格、进度条、树形结构等丰富的视觉元素，从而使你的命令行应用程序或日志更加直观和易于理解。

如果把终端比作一张白纸，`print()`函数只能用铅笔在上面写字，而Rich则是给了你一整套彩色画笔、尺子和各种绘图工具，让你能够在这张“纸”上创作出精美的作品。



1.1. Rich 库能做什么？

在介绍Rich的功能前，我们先来看看它解决了哪些实际问题：

1. **调试复杂数据结构**：当你需要查看嵌套字典或大型JSON时，普通的打印输出往往难以阅读。
2. **创建CLI界面**：命令行工具需要清晰的用户界面来呈现信息和交互元素。
3. **日志美化**：让错误、警告和信息日志通过不同颜色和样式区分，提高可读性。
4. **展示进度**：长时间运行的任务需要实时反馈进度。
5. **表格数据展示**：结构化数据需要整齐的表格形式来呈现。

1.2. Rich的安装

使用pip安装Rich非常简单，直接pip即可：

```
pip install rich
```

运行项目并下载源码

2. Rich的核心功能

现在让我们深入了解Rich的主要功能，并通过具体代码示例来学习如何使用它们。

2.1. 丰富的文本样式

Rich允许你轻松地文本添加颜色、背景色、粗体、斜体等样式：

```
1 from rich import print as rprint
2 from rich.console import Console
3 from rich.text import Text
4
5 # 使用增强版print函数
6 rprint("这是[bold red]红色粗体[/bold red]和[blue]蓝色[/blue]文本")
7
8 # 使用Console对象
9 console = Console()
10 console.print("这是", "一段", "文本", style="bold green")
11
12 # 使用Text对象构建复杂文本
13 text = Text("这是一段复杂的文本。")
14 text.stylize("bold", 0, 4) # 将"这是"设为粗体
15 text.stylize("italic blue", 5, 7) # 将"一段"设为斜体蓝色
16 console.print(text)
```

运行项目并下载源码



这种样式控制非常直观，就像在 **HTML** 中使用标签一样，使用[style_name]和[/style_name]来包围需要设置样式的文本。

这是**红色粗体**和**蓝色**文本
这是 **一段** 文本
这是一段**复杂的**文本。

2.2. 结构美化打印

当你处理复杂的Python对象时，像下面这样没有换行的json 格式文件，Rich能够智能地 **格式化** 它们：

```
1 from rich import print as rprint
2
3 # 美化字典打印
4 data = {"name": "Rich库示例", "version": 1.0, "features": ["格式化", "颜色", "表格", "进度条"], "details": {
5     "author": "Will McGugan", "license": "MIT", "contributors": ["张三", "李四", "王五"]
6 }}
7
8
9 rprint(data) # 自动格式化嵌套字典
10
11 # 美化对象打印
12 class Person:
13     def __init__(self, name, age):
14         self.name = name
15         self.age = age
16         self._private = "这是私有属性"
17
18 person = Person("张三", 25)
19 rprint(person) # 智能展示对象属性
```

运行项目并下载源码



Rich会自动 **识别数据** 类型，为不同层级应用缩进和颜色，使复杂结构变得一目了然。这对于调试来说尤其有用。

```
{
    'name': 'Rich库示例',
    'version': 1.0,
    'features': ['格式化', '颜色', '表格', '进度条'],
    'details': {'author': 'Will McGugan', 'license': 'MIT', 'contributors': ['张三', '李四', '王五']}
}
<__main__.Person object at 0x10260b290>
```

2.3. 表格展示

Rich提供了强大的表格功能，可以轻松创建格式化的表格：

```
1 from rich.console import Console
2 from rich.table import Table
3 from rich.text import Text
4
5 console = Console()
6
7 # Create a table
8 table = Table(title="Python Programming Language Features")
9
10 # Add columns
11 table.add_column("Feature", style="cyan", justify="right")
12 table.add_column("Description", style="magenta")
13 table.add_column("Example", style="green", max_width=20)
14
15 # Add rows
16 table.add_row("Dynamic Typing", "Variable types are determined at runtime", "x = 10\nx = 'text'")
17 table.add_row("List Comprehension", "Concisely create lists", "[x for x in range(10)]")
18 table.add_row("Decorator", "Syntax sugar", "@property\ndef name(self): ")
19
20 console.print(table)
```

运行项目并下载源码



这段代码会生成一个漂亮的三列表格，每列有不同的颜色，并且自动处理文本换行，如果用传统方法，你需要手动计算每一列的宽度和间距，而Rich为你处理了所有这些复杂性。

Python Programming Language Features		
Feature	Description	Example
Dynamic Typing	Variable types are determined at runtime	x = 10 x = 'text'
List Comprehension	Concisely create lists	
Decorator	Syntax sugar	@property def name(self):

2.4. 进度条

Rich的进度条功能让你可以实时展示任务进度，这个可以说是最常用的功能了。

```

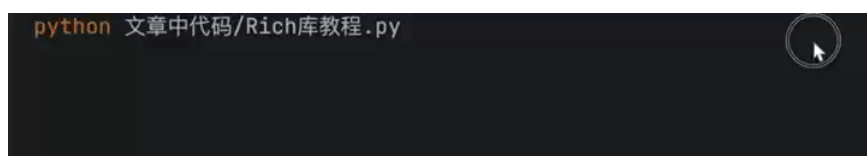
1 from rich.progress import Progress, TextColumn, BarColumn, TimeElapsedColumn
2 import time
3
4 # 创建自定义进度条
5 with Progress(
6     TextColumn("[bold blue]{task.description}"),
7     BarColumn(),
8     TextColumn("[progress.percentage]{task.percentage:>3.0f}%"),
9     TimeElapsedColumn()
10 ) as progress:
11     # 添加任务
12     task = progress.add_task("[red]正在处理...", total=100)
13
14     # 模拟工作流程
15     for i in range(100):
16         # 做一些工作
17         time.sleep(0.1)
18         # 更新进度
19         progress.update(task, advance=1)

```

运行项目并下载源码



这个进度条不仅显示完成百分比，还显示已用时间，让用户能够清楚地了解任务进展。



2.5. 日志记录

Rich可以替代标准日志模块，为日志添加颜色和格式：

```

1 import logging
2 from rich.logging import RichHandler
3
4 # 配置日志
5 logging.basicConfig(
6     level="INFO",
7     format="%(message)s",
8     datefmt="[%X]",
9     handlers=[RichHandler(rich_tracebacks=True)]
10 )
11
12 # 获取日志器
13 log = logging.getLogger("rich")
14
15 # 记录不同级别的日志
16 log.info("这是一条信息")
17 log.warning("这是一条警告")
18 log.error("这是一条错误")
19 try:
20     1/0
21 except Exception:
22     log.exception("发生了一个异常")

```

运行项目并下载源码



使用Rich的日志处理器后，不同级别的日志会以不同颜色显示，而且异常堆栈跟踪也会被美化，使错误更容易定位。

```
[14:25:17] INFO      这是一条信息
WARNING    这是一条警告
ERROR      这是一条错误
ERROR      发生了一个异常

Rich库教程.py:16
Rich库教程.py:17
Rich库教程.py:18
Rich库教程.py:22

Traceback (most recent call last)

/Users/lingxiaotian/工作空间/开发库/My
program/Python项目/文章中代码/Rich库教程.py:20 in <module>

17 log.warning("这是一条警告")
18 log.error("这是一条错误")
19 try:
20 | 1/0
21 except Exception:
22 | log.exception("发生了一个异常")
23

ZeroDivisionError: division by zero
```

2.6. 树形结构

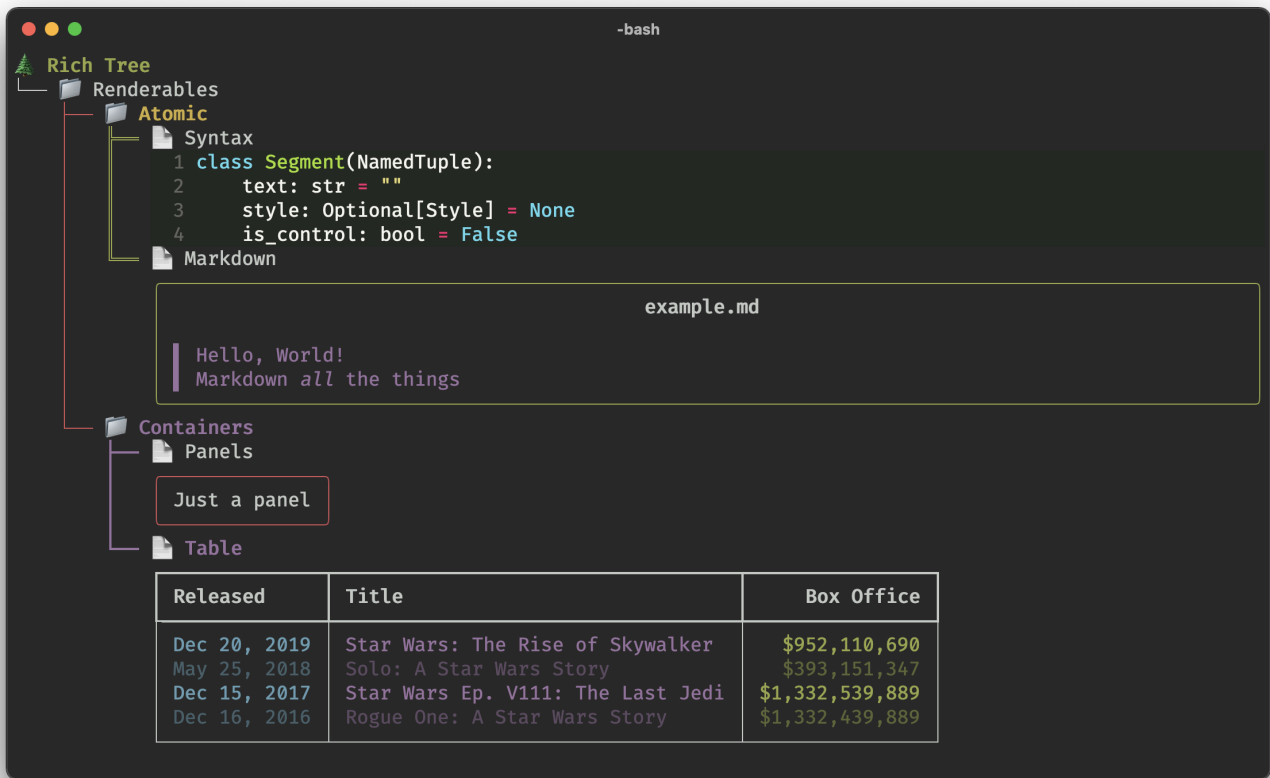
当你需要展示 **项目** 层级关系时，Rich的树形结构非常有用：

```
1 from rich import print
2 from rich.tree import Tree
3
4 # 创建树
5 tree = Tree("📁 项目目录")
6
7 # 添加分支和叶子
8 src = tree.add("📁 src")
9 src.add("📄 main.py")
10 src.add("📄 utils.py")
11
12 tests = tree.add("📁 tests")
13 tests.add("📄 test_main.py")
14 tests.add("📄 test_utils.py")
15
16 docs = tree.add("📁 docs")
17 docs.add("📄 index.md")
18 docs.add("📄 api.md")
19
20 # 打印树
21 print(tree)
```

运行项目并下载源码



这段代码会生成一个直观的目录树结构，使项目文件组织一目了然。



2.7. 面板和分栏

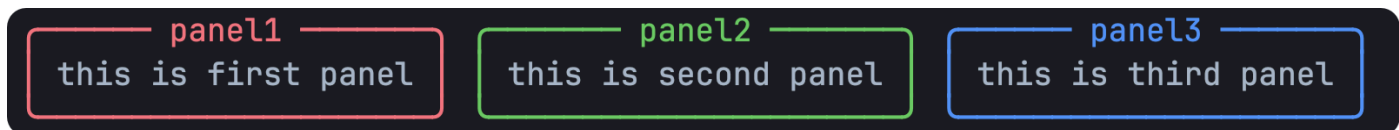
Rich可以创建带边框的面板，以及并排的分栏：

```
1 from rich.console import Console
2 from rich.panel import Panel
3 from rich.columns import Columns
4
5 console = Console()
6
7 # 创建面板
8 panel1 = Panel("this is first panel", title="panel1", border_style="red")
9 panel2 = Panel("this is second panel", title="panel2", border_style="green")
10 panel3 = Panel("this is third panel", title="panel3", border_style="blue")
11
12 # 使用分栏并排显示
13 console.print(Columns([panel1, panel2, panel3]))
```

运行项目并下载源码



这种布局功能让你可以在终端中创建类似仪表盘 的信息展示界面。



3. Rich的高级用法

除了上面介绍的基本功能外，Rich还有一些高级用法：

3.1. 自定义样式

你可以定义自己的样式组合：

```

1 from rich.console import Console
2 from rich.theme import Theme
3
4 # 自定义主题
5 custom_theme = Theme({
6     "info": "dim cyan",
7     "warning": "magenta",
8     "danger": "bold red underline"
9 })
10
11 # 使用自定义主题创建控制台
12 console = Console(theme=custom_theme)
13
14 # 使用自定义样式
15 console.print("这是一条信息", style="info")
16 console.print("这是一条警告", style="warning")
17 console.print("这是一条危险提示", style="danger")

```

运行项目并下载源码



这是一条信息
这是一条警告
这是一条危险提示

2. 跟踪函数执行

Rich可以帮助你跟踪函数执行时间和上下文:

```

1 from rich.console import Console
2 from rich.traceback import install
3
4 # 安装Rich的异常处理器
5 install()
6
7 console = Console()
8
9 # 装饰你想跟踪的函数
10 @console.status("[bold green]正在计算...")
11 def complex_calculation(n):
12     import time
13     time.sleep(n) # 模拟复杂计算
14     return n * n
15
16 # 执行并显示状态
17 result = complex_calculation(3)
18 console.print(f"计算结果: {result}")

```

运行项目并下载源码



```

Traceback (most recent call last)
/Users/lingxiaotian/工作空间/开发库/My program/Python项目/文章中代码/Rich库教程.py:10 in
<module>

      7 console = Console()
      8
      9 # 装饰你想跟踪的函数
> 10 @console.status("[bold green]正在计算...")
     11 def complex_calculation(n):
     12     import time
     13     time.sleep(n) # 模拟复杂计算

```

TypeError: 'Status' object is not callable

3. 输出到文件

Rich的输出不仅可以显示在终端，还可以保存到文件:


```

1 from rich.console import Console
2 from rich.table import Table
3
4 # 创建一个控制台，输出到文件
5 console = Console(record=True)
6
7 # 创建表格
8 table = Table(title="数据报告")
9 table.add_column("ID")
10 table.add_column("名称")
11 table.add_column("值")
12
13 # 添加数据
14 for i in range(10):
15     table.add_row(str(i), f"项目{i}", f"{i*100}")
16
17 # 打印表格
18 console.print(table)
19
20 # 保存到HTML文件
21 console.save_html("report.html")
22
23 # 保存为文本文件
24 with open("report.txt", "w", encoding="utf-8") as file:
25     file.write(console.export_text())

```

运行项目并下载源码



4. 小结

Rich库让Python开发者能够轻松创建美观、丰富且信息丰富的终端输出。

🔪 从简单的彩色文本，到复杂的实时更新仪表盘，Rich提供了一套完整的工具，让你的命令行获得前所未有的表现力。

参考文档 📄: [GitHub - Textualize/rich: Rich is a Python library for rich text and beautiful formatting in the terminal.](https://github.com/textualize/rich)

希望这篇教程能够帮助你掌握Rich库 🍷，并在日常编程中充分发挥它的潜力，好的工具不仅能提高效率，还能让编程过程变得更加愉快！